Ben 10 · Clareira LIVE

Relatório técnico — V43.8

Arquitetura, problemas da IA local e da voz, estrutura do código e plano de melhorias.

Data: 08/11/2026 · Referência de data: America/Sao_Paulo. Plataforma alvo informada: Realme GT3, 16 GB de RAM, 1 TB de armazenamento, Android com Termux. Versão do jogo: 43.8.0. Este relatório acompanha o projeto completo, não apenas o reparo incremental.

1. Situação desta entrega

O pacote reúne o jogo web, cenário e modelos 3D existentes, painel da LIVE, conectores, servidor Node, integração com IA local, motores de voz já existentes, instaladores, testes e documentação. A lógica de conversa é a mesma V43.8 publicada na atualização incremental. A voz selecionada no celular não é trocada por esta entrega.

Os pesos grandes de IA (GGUF, ONNX e checkpoints baixados no telefone), o ambiente Debian/proot, node_modules, chaves, .env real, progresso e caches privados não fazem parte do ZIP. Eles são instalados ou preservados separadamente. “Projeto completo” significa código e recursos distribuíveis completos; não significa uma cópia do armazenamento privado do telefone.

Área	Situação atual	Limite conhecido
Conversa local	Integração disponível, respostas diretas e modelo Qwen	Perguntas gerais ainda exigem inferência em CPU
Voz	Motores existentes e reprodução preservados	Timbre diferente e palavras enroladas continuam pendentes
Curtidas e chat	Correções de eventos e fila incluídas	Dependem de eventos realmente recebidos do TikTok
Encerramento no Termux	Novo bug registrado como LOCAL-010	Causa inicial não confirmada; sem log da última queda

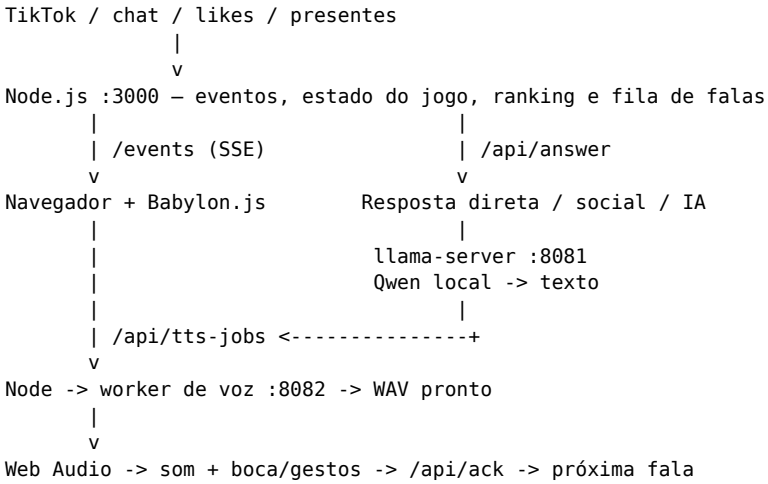
Não foi feita transmissão prolongada em um GT3 físico nesta nuvem. Nenhum resultado de Linux substitui essa validação. O novo relato de encerramento espontâneo foi documentado, não declarado resolvido.

Como interpretar as evidências

“Relatado” identifica o que o usuário observou. “Confirmado no código” descreve comportamento que a revisão permite demonstrar. “Validado em teste” se refere ao ambiente e aos limites citados. “Hipótese” exige coleta adicional. Um WAV válido não comprova semelhança vocal; uma legenda correta não comprova reprodução de áudio.

2. Arquitetura e fluxo de execução

O jogo tem três serviços locais principais. O navegador desenha o cenário 3D e toca o áudio; Node coordena a LIVE e as filas; dois serviços separados geram texto e voz.



As portas 3000, 8081 e 8082 são os padrões; a configuração permite mudar as portas. Os serviços de IA usam HTTP em 127.0.0.1, sem precisar de chave de API. Os conectores TikTok, fotos e player de música continuam dependendo de internet.

2.1 Da interação à resposta

- server/live.js e live-alternative.js recebem os eventos dos conectores; live-events.js normaliza os dados. A origem TikTok permanece separada da demonstração local.
- server/core.js trata deduplicação, pontos, ranking e SpeechQueue. Presentes recebem prioridade; a fala já em reprodução termina antes de outra começar.
- AISession escolhe o provedor. Roteiros, saudações e respostas diretas são caminhos distintos de uma consulta real ao modelo.
- quick-answer.js calcula apenas contas simples reconhecidas e responde um conjunto limitado de fatos verificados. Perguntas compostas ou fora desse conjunto seguem para o modelo.
- LocalTextClient monta o contexto clássico, estado relevante do jogo e, quando necessário, até duas interações recentes da mesma pessoa. O Qwen gera a fala; não controla diretamente comandos do sistema.
- O navegador solicita a voz, acompanha a tarefa e reproduz o WAV. SpeechConsumer só confirma a pergunta após o término da reprodução. Falhas de voz preservam a mensagem para nova tentativa.

2.2 IA de conversa

O perfil gt3 usa unsloth/Qwen3-4B-Instruct-2507-GGUF, quantização Q4_K_M. O modelo é servido por llama.cpp com API compatível com chat/completions. O nome ben-classico-local é um alias do serviço; não é um modelo treinado exclusivamente como Ben 10.

O processo usa contexto de 4096 tokens e quatro threads na configuração atual. O pedido limita a geração a 96 tokens, desativa o raciocínio exposto e pede até duas frases curtas em português brasileiro. A personalidade vem das instruções e das referências de server/classic.js. Isso não garante conhecimento completo de todos os episódios nem ausência de erros factuais.

A V43.8 prepara o prefixo do contexto ao iniciar e permite seu reaproveitamento no llama.cpp. Respostas gerais independentes podem ficar em cache por até 120 segundos, com limite de 64 entradas.

Perguntas pessoais, seguimentos e estado/tempo não usam esse cache. O teste de IA força uma consulta real; uma saudação pronta não certifica que o modelo está online.

2.3 Voz e clonador existentes

Conversa e voz são independentes. A frase final segue para o motor selecionado em install.json. O pacote não ativa um motor novo nem executa novamente a preparação do clone.

Caminho de voz	Funcionamento	Consequência
Piper	Texto para voz neural pt-BR com ONNX	Voz original do modelo, sem copiar uma referência
OpenVoice V2	Piper Cadu produz a fala e OpenVoice converte o timbre usando a referência	Pronúncia e timbre dependem de etapas diferentes; conversão pode degradar clareza
Qwen3-TTS nativo	Biblioteca nativa, modelos de voz/tokenizer e referência local	Geração pesada em CPU; houve SIGILL e demora no histórico

openvoice_voice.py usa duas threads para PyTorch e duas para ONNX. Os modelos e o embedding da referência são carregados pelo worker. Isso não é uma clonagem idêntica ou um treinamento completo da dublagem original. O papel de cada motor precisa aparecer no diagnóstico, sem rotular Piper como clone.

voice_jobs.py permite uma síntese ativa por vez, identifica tarefas e mantém WAVs prontos. Há limite de 32 tarefas e cerca de 24 MB para o cache de áudio, além de expiração em 30 minutos. Esses limites não limitam a RAM ocupada pelos pesos de PyTorch, ONNX ou llama.cpp.

local-voice-jobs.js consulta o andamento aproximadamente a cada 900 ms. A fala pode ser acompanhada por até dez minutos, em vez de ser descartada por uma única requisição HTTP curta. Isso evita parte do silêncio causado por timeout, mas não acelera a síntese. O fluxo atual espera o WAV completo; não é voz transmitida em pequenos trechos enquanto nasce.

neural-audio.js decodifica e toca o áudio via Web Audio, com atualização da boca. O primeiro toque do usuário libera o contexto de áudio no navegador. Suspensão, troca de aplicativo ou perda do gesto de ativação podem exigir novo toque em ♫.

2.4 Processos e inicialização

local-ai.sh start chama restart.js com verificação da instalação e da versão. Ele só encerra processos cuja origem consegue confirmar. Portas ocupadas por processos externos são recusadas, sem encerramento indiscriminado.

run.js inicia llama-server, o worker de voz e o servidor do jogo. Espera os serviços ficarem prontos, prepara o contexto e mantém a sessão ativa. Na implementação atual, a saída de qualquer serviço filho monitorado provoca erro e limpeza dos demais processos. Esse é um mecanismo de encerramento conjunto confirmado no código; não prova por que o primeiro serviço saiu no telefone.

O navegador usa um bloqueio de tela quando disponível. Isso é diferente de um bloqueio de CPU do Android e não assegura que o sistema manterá o Termux vivo em segundo plano.

3. Estrutura do código e dos dados

```
game/
  public/                 Interface, cenário, animações e áudio web
    arena-app.js          Coordenação da tela, fila e controles
    world.js              Renderização e personagens
    camp-world.js          Clareira e acampamento
    neural-audio.js        Decodificação e reprodução
    speech-consumer.js     Resposta -> voz -> confirmação
    local-voice-jobs.js    Acompanhamento das tarefas de voz
    ben-dialogue.js        Tratamento do nome e da fala
    spoken-number.js       Números calculados por extenso
    music-ui.js            Pedidos e player de música
  server/
    index.js               HTTP, SSE e coordenação do jogo
    core.js                Eventos, ranking e fila
    ai-session.js          Seleção do provedor e memória curta
    local-ai.js            Clientes HTTP da IA e da voz locais
    quick-answer.js        Respostas verificadas sem inferência
    classic.js             Referências do Ben clássico
    conversation.js        Resposta e falhas do provedor
    live*.js               Conectores, público e recepção
    chat-goals.js          Metas coletivas do chat
    alien-unlocks.js       Progresso e desbloqueio de DNA
    music.js               Fila de pedidos musicais
  scripts/local-ai/
    install.sh             Preparação do ambiente local
    run.js                 Serviços e ciclo de vida
    restart.js             Reinício com conferência de origem
    project-processes.js   Identificação dos processos do projeto
    download_models.py     Catálogo, download e integridade
    tts_server.py          Servidor HTTP de voz
    voice_jobs.py          Tarefas e cache WAV
    openvoice_voice.py     Conversor já integrado
    clone_voice.py         Interface do motor Qwen3-TTS existente
    clone_native.py        Compilação e perfil de instruções
  assets/                  Recursos distribuíveis e referência existente
  docs/                    Documentação e este relatório
  tests/                   Testes de unidade e integração
  reports/v43_8/           Evidências desta versão
  third_party/             Licenças e dependências incorporadas
  package.json             Comandos e dependências Node
  start.sh / local-ai.sh   Entradas de execução
  update.sh                Aplicação do ZIP completo
```

Onde ficam as informações privadas

Local	Conteúdo	Incluído no ZIP público?
game/data/	Configuração, progresso, modelos personalizados e chaves locais	Não
game/.env	Variáveis e chaves fornecidas pelo operador	Não
~/.local/share/ben10-local-ai/	install.json, pesos, referência selecionada, caches e logs	Não
Ambiente do proot-distro	Python e dependências de inferência	Não
game/assets e public	Recursos distribuídos com o jogo, incluindo referência/teste já existentes	Sim

BEN10_LOCAL_ROOT permite escolher outra raiz local. A mesma variável deve ser usada na instalação e na inicialização. Portas e seleção de provedor ficam na configuração do jogo. Não copie chaves para

public/ nem para o relatório.

Os reparos avulsos com V43_7 no nome são arquivos históricos. Não devem ser executados sobre a V43.8, pois podem reverter arquivos. Use update.sh para o pacote completo. Documentos antigos em docs/ também retratam suas respectivas versões; este relatório descreve a configuração atual.

4. Registro dos bugs — IA e instalação

LOCAL-001 · Legenda aparece, mas a fala não é ouvida

Estado: falha histórica com correções no transporte/reprodução; verificar novamente se houver recorrência. Impacto: alto.

Evidência: o usuário conseguiu ouvir preview.wav em um player externo, enquanto o jogo mostrava a legenda e permanecia em “preparando voz”. Isso demonstrou que pelo menos aquele arquivo era compreensível; não demonstrou que a nova fala chegou ao navegador. Em outro registro, os serviços estavam offline.

Camadas possíveis: geração demorada, serviço parado, tarefa perdida, decodificação ou AudioContext suspenso. A implementação atual acompanha tarefas, mantém a fala em caso de erro, separa a legenda da conclusão do áudio e pede toque quando o navegador bloqueia a reprodução. Testes com tom sintético validam esse caminho; não certificam o timbre.

LOCAL-002 · Demora grande para responder

Estado: parcialmente melhorado; aberto para desempenho no celular. Impacto: alto.

Evidência do telefone: um log Qwen3-TTS enviado pelo usuário registra 69,548 s de geração para 5,10 s de áudio, com RTF 13,645. Esse exemplo era do motor nativo naquele momento; não é uma medição do OpenVoice atual nem de todas as falas.

A espera total soma fila, modelo de texto, geração/conversão de voz e reprodução. A V43.8 reduz espera de texto com respostas diretas limitadas, contexto estável, cache, frases curtas e menos histórico sem relação. Avisos automáticos antigos expiram. Essas mudanças não tornam instantâneo um sintetizador lento. Medir cada etapa com a LIVE e o cenário ativos no GT3 continua necessário.

LOCAL-003 · Palavras enroladas, troca percebida de R/L e timbre diferente

Estado: aberto; não corrigido nesta entrega. Impacto: alto.

Evidência: o usuário descreveu troca de R por L, depois palavras enroladas e voz distante do personagem. A configuração informada era pt-br e tinha IDs distintos para fonemas de R e L. Isso afasta a hipótese simples de ambos usarem o mesmo ID, mas não identifica a origem acústica do defeito.

No OpenVoice, a origem pode estar na fala base, no áudio de referência ou na conversão do timbre; no Qwen, no condicionamento e na própria síntese. Não aplicar substituição textual global de R por outra letra. A V43.8 envia resultados de contas por extenso, mas não conserta o timbre. Próximo critério: comparar a mesma frase na saída base e final, com avaliação humana, duração e taxa de palavras compreensíveis. Uma voz neural original mais clara é alternativa se o operador preferir.

LOCAL-004 · “Oi”, “olá” e perguntas sem resposta ou fora do assunto

Estado: caminhos corrigidos em testes; LIVE real ainda exige validação. Impacto: alto.

As saudações completas recebem resposta social pertinente. Saudação junto de pergunta continua sendo tratada como pergunta. O provedor local não exige chave Gemini; erro de conexão deve mostrar a falha local. A pergunta só sai da fila após o áudio terminar. A V43.8 limita referências irrelevantes e elimina prefixos repetidos com o nome da pessoa. Perguntas arbitrárias ainda podem receber erros do

modelo; não existe garantia de precisão universal.

LOCAL-005 · Características da voz eram lidas antes da frase

Estado: correção já incorporada ao fluxo de fala; manter regressão. Impacto: médio.

Descrições de atuação e instruções do personagem pertencem ao gerador de texto, não ao texto enviado ao sintetizador. O caminho atual passa a frase final, limpa marcações e pede somente a fala ao modelo. Verificar a carga enviada ao motor antes de atribuir esse problema ao vocoder. Alterações futuras no prompt não devem reintroduzir rubricas na fala.

4. Registro dos bugs — infraestrutura e eventos

LOCAL-006 · APT: NO_PUBKEY 5A897D96E57CF20C

Estado: contornado no instalador da IA com a fonte principal configurada e verificada. Impacto: bloqueava a instalação.

O usuário tinha termux.sources (Deb822), glibc.list e tur.sources. A consulta isolada ao termux.sources existente, apontando para termux.net, terminou com sucesso. A falha não justificava ignorar assinaturas. O instalador deve usar a fonte principal real, preservar repositórios extras e só consultar APT quando faltarem ferramentas. Caso volte, conferir edição do Termux, fonte e chave correspondente; não usar trusted=yes ou allow-unauthenticated.

LOCAL-007 · Download do modelo retornou HTTP 401

Estado: seleção do catálogo público corrigida no histórico. Impacto: bloqueava a IA local.

A consulta feita no aparelho confirmou o repositório unsloth/Qwen3-4B-Instruct-2507-GGUF e o arquivo Qwen3-4B-Instruct-2507-Q4_K_M.gguf. O downloader consulta metadados, seleciona arquivos e verifica integridade. Um 401 isolado não prova que todo acesso ao Hugging Face precisa de chave. Se repetir, guardar URL final sem credenciais, status e nome do arquivo; verificar alteração do catálogo ou política do provedor antes de reinstalar tudo.

LOCAL-008 · SIGILL no preview do clonador nativo

Estado: perfil conservador implementado; ocorrência original não reproduzida no GT3 nesta nuvem. Impacto: encerrava o processo de voz.

O aparelho informou GGML_NATIVE=ON e arquitetura ARM vazia no cache de compilação. O reparo existente desativa otimização nativa e extensões opcionais e usa ARMv8-A, conferindo os comandos de compilação. SIGILL significa instrução inválida; sem a instrução/endereço da falha, não certifica sozinho a causa. Preservar modelos e recompilar somente o motor quando necessário. Não confundir esse sinal histórico com a nova queda sem log.

LOCAL-009 · Curtidas não contabilizadas

Estado: correção de deduplicação incluída; dependência do conector permanece. Impacto: alto para missões e desbloqueios.

IDs vazios ou zero não podem fazer todos os lotes seguintes parecerem duplicados. A V43.7 corrigiu esse caso, preservado na V43.8. Eventos válidos repetidos continuam sendo deduplicados. Lotes não são cortados em mil antes de chegar ao progresso. O teste controlado com 17 + 23 curtidas resultou em 40; isso não mede eventos que o TikTok deixou de entregar. Conferir likesReceived, lastLikeAt, origem demo/TikTok e conexão antes de alterar a barra de DNA.

LOCAL-011 · Nomes duplicados e referências fora do assunto

Estado: corrigido em casos de regressão V43.8. Impacto: médio.

No teste inicial o histórico levava o modelo a repetir o nome. O servidor agora remove a chamada nominal antes de guardar a resposta e acrescenta o nome uma vez na saída. Perguntas independentes dispensam o histórico anterior e a recuperação de referências filtra termos genéricos. O teste posterior com modelo real respondeu sobre fotossíntese e admitiu não saber o aniversário da pessoa. Esses exemplos não constituem avaliação ampla do modelo.

LOCAL-012 · Interface, servidor e voz de versões diferentes

Estado: detecção e reinício existentes; atenção ao atualizar arquivos com processos ativos. Impacto: alto.

A URL com ?v=43.8.0 não muda sozinha o código já carregado no Node ou no worker. O projeto compara BUILD_ID, cabeçalhos e saúde dos serviços. restart.js tenta reiniciar somente processos confirmados da instalação. Manter a sessão original aberta e iniciar pela pasta correta; não abrir várias cópias ocupando as mesmas portas. Um serviço parado não deve ser apresentado como falha de chave de API.

5. Novo bug: LOCAL-010 · O programa para sozinho no Termux

Prioridade: P0 de diagnóstico. Estado: relatado pelo usuário, ainda sem reprodução controlada e sem log da última ocorrência.

Descrição recebida: “Em determinado tempo de live o programa simplesmente para no Termux sozinho”. Não foram informados tempo até a falha, sinal, código de saída, motor selecionado ou comportamento da tela no momento da parada. O bug está aberto; não há correção comprovada nesta entrega.

Comportamento confirmado no código

run.js acompanha os processos iniciados de conversa, voz e jogo. Se um deles sai, o laço final gera erro; o catch registra a falha em logs/inicializacao.log e chama cleanup(), que envia SIGTERM aos demais processos conhecidos. Assim, um defeito isolado pode encerrar toda a sessão. O iniciador não faz recuperação automática limitada por serviço.

O supervisor já registra saídas de filhos com código/sinal e pedidos de SIGINT/SIGTERM. Se o Android matar o próprio supervisor com SIGKILL, ele não poderá executar esse registro ou a limpeza. A ausência de uma última linha não prova qual mecanismo do Android atuou.

Hipótese a investigar	Evidência que ajudaria	Não afirmar sem evidência
Serviço de voz ou LLM saiu primeiro	Última saída de filho e log do serviço no mesmo horário	Que o clonador causou toda queda
Falta de memória/limites do Android	SIGKILL, RAM antes da queda, registro de encerramento do sistema	Que 16 GB elimina risco de pressão de RAM
Bateria/segundo plano	Queda após minimizar, bloquear tela ou política de economia	Que um bloqueio de tela do navegador protege o Termux
Falha nativa	SIGILL/SIGSEGV e log/backtrace do processo	Que o SIGILL antigo explica a nova ocorrência
Exceção JavaScript ou erro de processo	Stack trace no terminal e código de saída	Que foi desconexão do TikTok apenas
Aquecimento e carga prolongada	Temperatura e latências ao longo da sessão	Que temperatura sozinha matou o programa

Dados mínimos da próxima ocorrência

Registrar horário local, minutos de LIVE até parar, se o aparelho estava bloqueado ou em outro aplicativo, motor de voz ativo e texto final do Termux. Antes de iniciar outra sessão, guardar os trechos finais de inicializacao.log, conversa.log e voz.log. O diagnóstico da interface distingue fila, modelo, geração da voz e reprodução enquanto o servidor ainda responde.

O código de saída deve ser capturado imediatamente após o comando terminar. O valor 137 pode corresponder a SIGKILL, porém não identifica quem enviou o sinal nem prova sozinho falta de memória. SIGKILL não pode ser tratado por um handler dentro do processo morto.

Critério de encerramento deste bug: identificar o primeiro processo/evento que falha, reproduzir sob carga semelhante, aplicar correção específica e repetir uma LIVE longa com evidências de continuidade, memória e recuperação. Reiniciar repetidamente sem diagnóstico pode ocultar o defeito.

6. Plano de melhorias recomendado

Prioridade	Trabalho proposto	Critério de aceitação
P0	Registrar vida do supervisor, filhos, PID, código/sinal e horário com rotação	A próxima parada permite identificar o primeiro componente que saiu
P0	Teste prolongado no GT3 com LIVE, cenário, Qwen e voz juntos	Sessão de 60–120 min com RAM, temperatura, tempos e resultado registrados
P1	Recuperação limitada de voz/texto, mantendo o jogo disponível	Falha simulada mostra aviso e retenta com limite; sem reiniciar processos externos
P1	Persistir a fila necessária para retomada segura	Reinício não perde pergunta aceita nem reproduz duas vezes uma fala já confirmada
P1	Medir e melhorar a clareza vocal	Frases fixas com R/RR, números e nomes compreensíveis; comparação cega entre versões
P1	Controlar saturação da fila e comprimento das respostas	Percentis de espera e tamanho da fila permanecem aceitáveis na carga alvo
P2	Avaliar modelo menor como opção, sem troca automática	Comparação de precisão, RAM, resposta e temperatura no mesmo aparelho
P2	Melhorar áudio em trechos com fila e cancelamento seguros	Primeiro áudio chega antes, sem sobreposição, palavras cortadas ou ACK antecipado
P2	Expandir base verificada do clássico com fontes	Questões de episódios/personagens em conjunto de avaliação; dúvidas admitidas
P2	Perfil gráfico para transmissão prolongada	FPS e resolução ajustados sem degradar fala, cenário ou leitura dos painéis
P3	Separar módulos grandes de interface e servidor	Alterar voz não exige mexer na coordenação de ranking/cenário
P3	Unificar documentos e arquivar reparos antigos	Instalação e atualização seguem uma única orientação por versão

Esses itens são próximos trabalhos; não foram apresentados como implementados nesta entrega. A recuperação automática precisa de limites e identificação dos processos para evitar ciclos de reinício e conflitos de portas. Um watchdog dentro do Termux também pode morrer quando o Android encerra o aplicativo inteiro.

Precisão e personalidade

Separar quatro objetivos na avaliação: responder ao conteúdo da pergunta, usar fatos corretos, soar como o Ben clássico nas falas e permanecer compreensível em áudio. Melhorar uma dessas medidas não garante as outras. Saudações e chamadas ao TAP TAP devem respeitar a fila; inserir um pedido de interação em toda resposta piora a pertinência.

Não prometer que um prompt dá ao modelo a identidade ou a memória completa do personagem. Priorizar respostas curtas, confirmação de dúvida, recuperação de fatos confiáveis e testes com comentários reais anonimizados.

7. Evidências e limites de validação

Modelo real de texto — V43.8

Máquina Linux x86_64, CPU, sem renderizar o cenário; mesmo Qwen3-4B-Instruct-2507 Q4_K_M. Não são tempos do Realme GT3 e não incluem síntese de voz.

Pergunta / etapa	Caminho	Tempo medido
Preparação do contexto	Modelo real, uma vez no início	7,243 s
Sete vezes oito	Resposta direta, resultado 56	33 ms
Quem criou o Omnitrix?	Resposta direta verificada	0 ms na resolução do relógio do teste
O que é fotossíntese?	Modelo real	13,819 s
Capital do Japão	Modelo real	6,649 s
Qual é meu aniversário?	Modelo real, admitiu não saber	10,112 s

Os pedidos gerais reutilizaram parte dos tokens de contexto. Não foi feito estudo estatístico de latência; são amostras pontuais. Não foi medida aceleração de voz nova nesta versão.

Testes funcionais e distribuição

A rodada anterior da V43.8 cobriu 229 testes de regressão, com o caso afetado repetido após trocar uma pergunta que passou a ter resposta direta. O navegador Chromium validou conta sem consulta ao modelo, números por extenso na solicitação de voz, resposta geral com serviço simulado, cache entre pessoas sem misturar seus nomes e confirmação somente após o som terminar.

O áudio do teste funcional é um tom sintético. O serviço de texto nesse teste de navegador é simulado. A medição do modelo real acima é um teste separado. Nenhum deles prova semelhança com a voz da dublagem ou estabilidade de uma LIVE prolongada no Android.

O instalador incremental foi verificado quanto a integridade, backup e preservação de dados em fixtures. A suíte completa desta distribuição passou nos 235 testes. O pacote tem uma verificação adicional própria de build e extração; o resultado fica em reports/v43_8/full-distribution.json, incluído no ZIP. Arquivos privados e pesos grandes devem ficar excluídos. O SHA-256 externo identifica o ZIP publicado.

Fontes deste relatório

- Relatos, comandos e logs trazidos pelo usuário nesta conversa: APT, catálogo Hugging Face, SIGILL, serviços offline, geração de 69,548 s, preview compreensível e voz enrolada.
- Revisão dos arquivos citados da V43.8, em especial run.js, restart.js, local-ai.js, voice_jobs.py e SpeechConsumer.
- reports/v43_8/text-benchmark.json, browser.json e update.json; testes recentes de distribuição no arquivo indicado acima.

Não foram copiados chaves, .env real, conversas privadas da LIVE ou arquivos de modelos do aparelho para este relatório. Nomes usados nos testes são fixtures.

8. Operação, atualização e diagnóstico

Atualizar uma instalação existente

Baixe BEN10_LIVE_V43_8_TERMUX.zip em Downloads. Use a pasta atual do jogo para preservar seu data/ e a referência dos processos. O atualizador mantém os dados locais e aplica o código do arquivo completo.

```
cd ~/ben10-v34/game &&
bash update.sh --archive \
  ~/storage/downloads/BEN10_LIVE_V43_8_TERMUX.zip &&
bash local-ai.sh start
```

Abra http://127.0.0.1:3000/?v=43.8.0 e toque em ♫. Mantenha a sessão do Termux aberta. A IA já instalada não precisa ser baixada novamente. A conexão com sua LIVE deve ser conferida no painel.

Nova instalação

Extraia o ZIP no armazenamento privado do Termux e execute bash install.sh na pasta game. Em um telefone sem IA instalada, use bash local-ai.sh install --model gt3 uma vez; essa etapa usa internet, vários GB e tempo de instalação. A voz não terá um timbre de referência automaticamente só porque o jogo foi extraído. Leia local-ai.sh help e os documentos da voz existente antes de alterar uma seleção que funciona.

Iniciar, conferir e parar

```
cd ~/ben10-v34/game
bash local-ai.sh start
```

Em outra sessão, quando necessário:

```
cd ~/ben10-v34/game
bash local-ai.sh status
bash local-ai.sh stop
```

Coletar o fim dos logs após uma queda

Faça isso antes de iniciar novamente, para facilitar a correlação. Não exponha .env ou chaves. Os logs podem conter nomes e textos de participantes; revise e anonimize antes de compartilhá-los.

```
cd ~/ben10-v34/game
bash local-ai.sh status
task_logs="${BEN10_LOCAL_ROOT:-$HOME/.local/share/ben10-local-ai}/logs"
tail -n 80 "$task_logs/inicializacao.log"
tail -n 80 "$task_logs/conversa.log"
tail -n 80 "$task_logs/voz.log"
```

Para observar o retorno do iniciador, execute em primeiro plano. O printf roda apenas quando o programa termina:

```
bash local-ai.sh start
task_exit=$?
printf 'Iniciador terminou com código %s\n' "$task_exit"
```

No jogo, LIVE → IA e voz → copiar diagnóstico informa versões, saúde dos serviços, etapa da fila e tempos da última fala. Modelo offline, clonador ocupado e navegador sem autorização de áudio são problemas diferentes e devem ser tratados separadamente.

O bloqueio de suspensão do Termux e a política de bateria do Android podem ser avaliados para o teste prolongado. Essas medidas não corrigem uma exceção, um SIGILL ou consumo excessivo de memória e não devem ser apresentadas como solução já comprovada do LOCAL-010.